Solution Architecture

Incident Lifecycle Automation in ServiceNow

1. Purpose

This document presents a high-level logical architecture for the proposed Incident Lifecycle Automation solution in ServiceNow. It describes the main actors, platform capabilities, records, and process relationships. It should be updated to match the actual configured instance.

2. Architecture components

- End user: reports an issue and checks permitted incident status information.
- ServiceNow user interface: incident form or approved portal through which users interact with the platform.
- Incident Management: maintains incident records, lifecycle state, priority, assignment, activity, and resolution details.
- Automation layer: Flow Designer and/or Business Rules support selected workflow behaviour where configured.
- Knowledge Management: stores searchable support articles used during investigation.
- Access control layer: roles and ACLs govern access to records and fields.
- Change Management: manages controlled changes that may be needed to resolve an incident.
- Reporting and review: authorised users can review available incident data and test evidence, subject to permissions and configured reports.

3. Logical architecture flow

- End user → ServiceNow interface → Incident Management record.
- Incident Management record ↔ Automation layer (selected assignment, notification, or lifecycle actions).
- Support agent ↔ Incident record (investigation, work notes, status, and resolution details).
- Support agent → Knowledge Management (search relevant articles).
- Incident requiring controlled change → Change Management (related change record, where applicable).
- Incident records → authorised reporting and review.

4. Data and security considerations

Use the standard incident record and fields available in the instance unless the requirements justify approved customisation.

Restrict incident data and work notes according to roles and ACLs.

Use sample or anonymised data for demonstrations and documentation.

Avoid exposing sensitive incident details in notifications or user-facing status messages.

Test automation for permissions, error paths, and expected lifecycle behaviour.

5. Environment and deployment approach

Design and configure in an authorised development or training instance.

Validate requirements and test cases before moving changes to another environment.

Use the organisation's approved update-set or deployment process if deployment is in scope.

Record dependencies such as application scope, roles, plugins, and platform release.

6. Architecture assumptions and limitations

This is a logical architecture, not a claim that every component has already been implemented.

Exact integrations, notifications, assignment rules, reports, and automation depend on the configured instance and project scope.

Production availability, performance, and measurable service improvements must be verified separately.

4. Logical Architecture Diagram

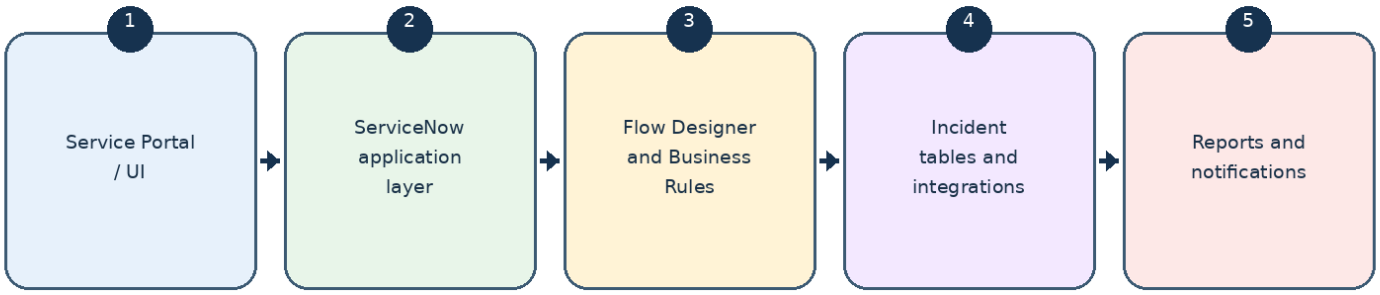
Conceptual view of the proposed components and their relationships.



Process Diagram

Solution Architecture

Incident Lifecycle Automation • Process diagram



Expected output: a traceable, testable process with clear ownership and documented outcomes.